Distributed Exchange System

Distributed Systems & Middleware Technologies

Slim Jabou & Ahmed Zarrad
University of Pisa

The Context

Modern real-time alert systems must process massive concurrent subscriptions under high load.

The Risks

Missed Alerts

Network partitions dropping critical threshold events.

Duplicate Triggers

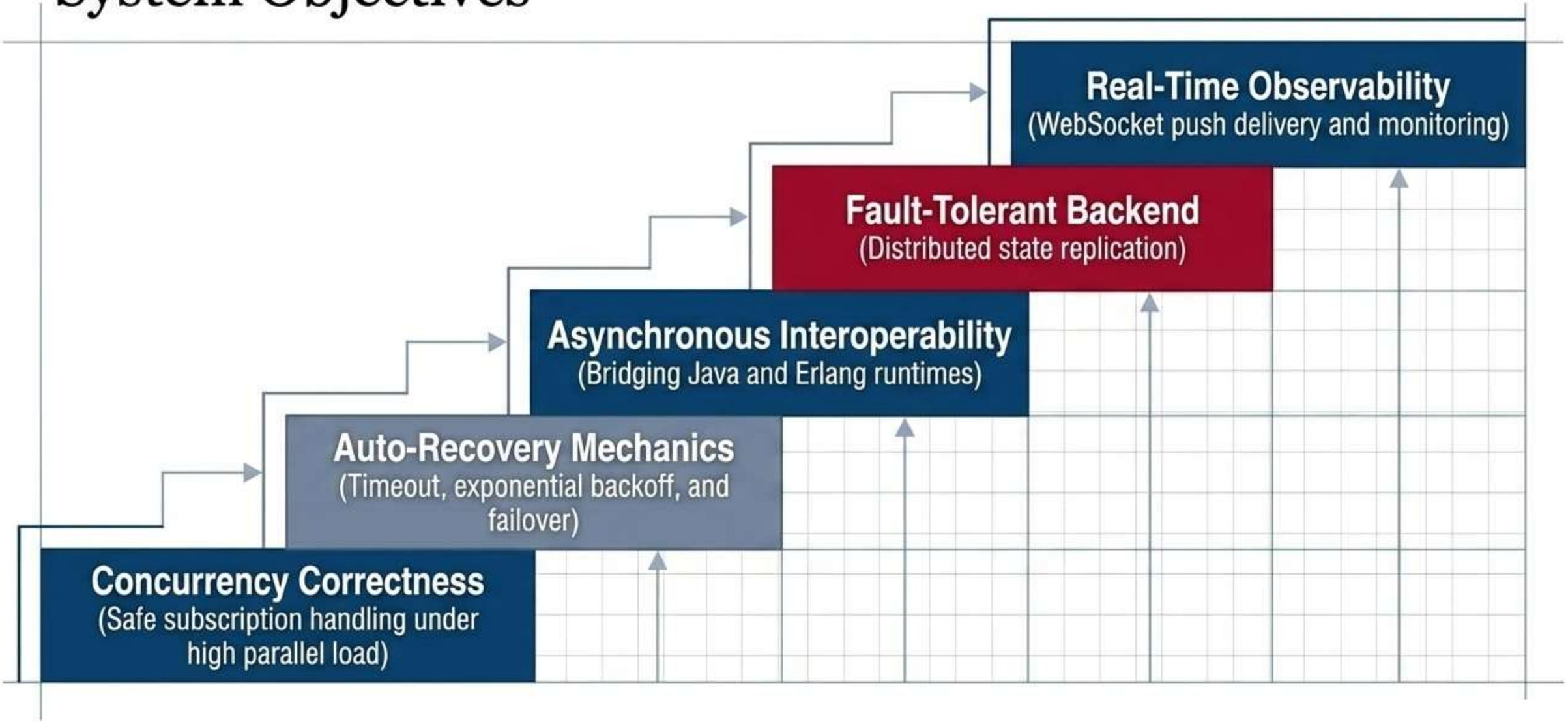
Race conditions causing overlapping alert deliveries.

Stale Routing

Dead network paths resulting in failed deliveries post-failover.

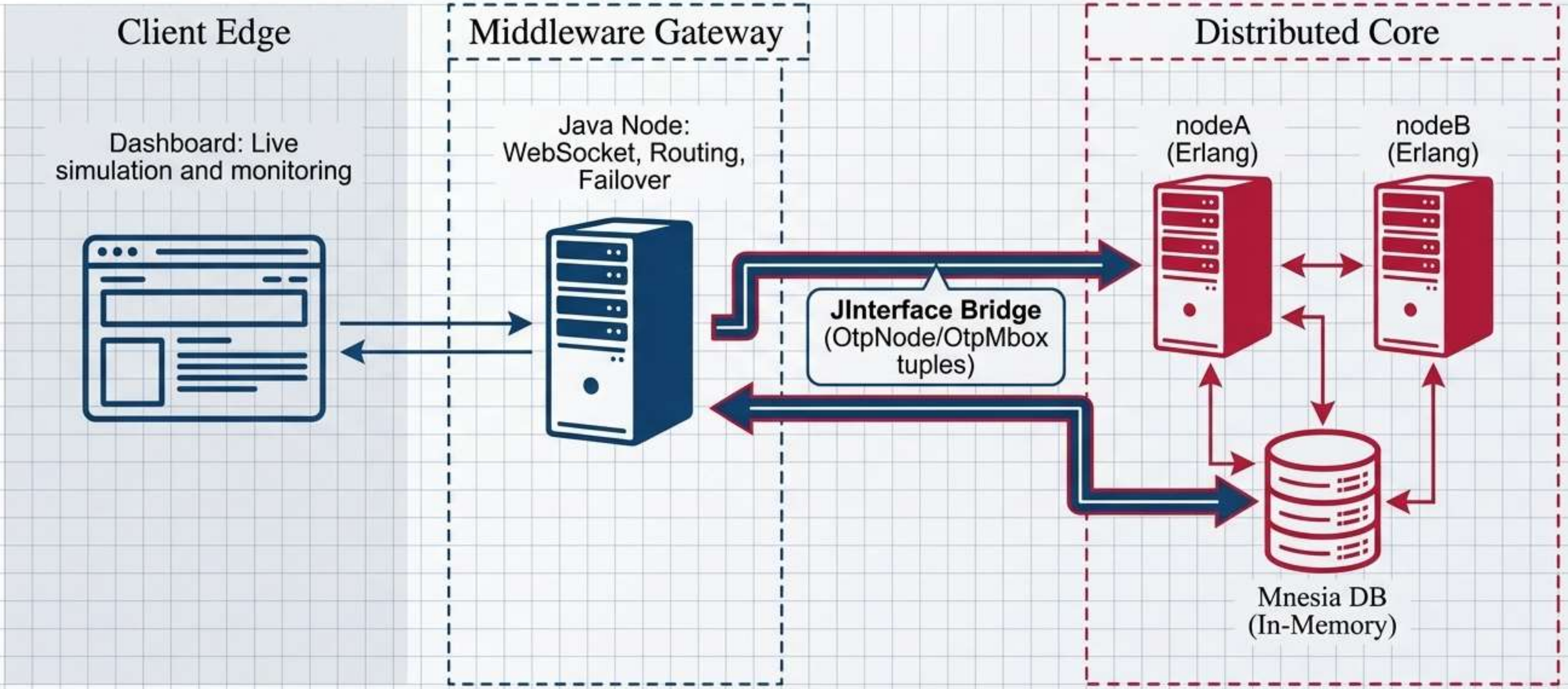
System Requirement: Flawless synchronization, automated fault handling, and consistent distributed state.

System Objectives



DIPARTIMENTO DI INGEGNERIA

High-Level Architecture Map

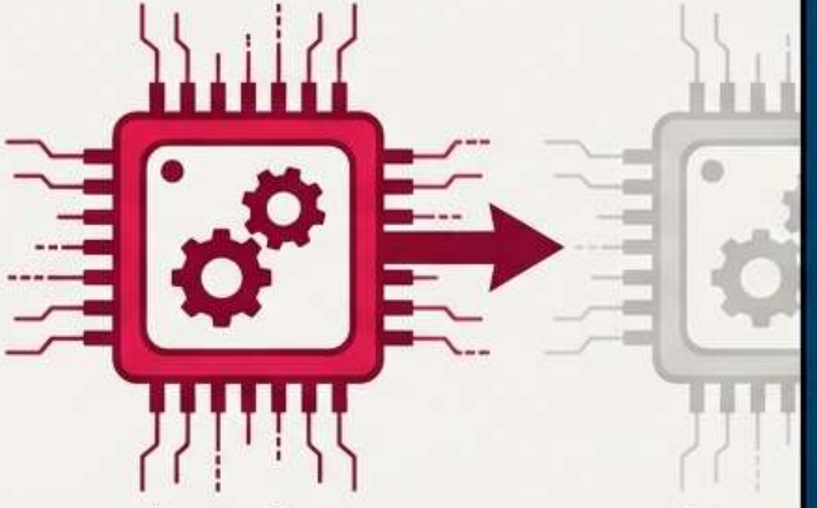


Mnesia Data Model

Table: subscriptions (Mode: ram_copies)	
client_pid	Java mailbox PID (Direct routing)
stock	Binary symbol (e.g., <<"TSLA">>)
threshold	Float target
raw_sub	Original command (Deduplication / UI ID)

Architectural Classification

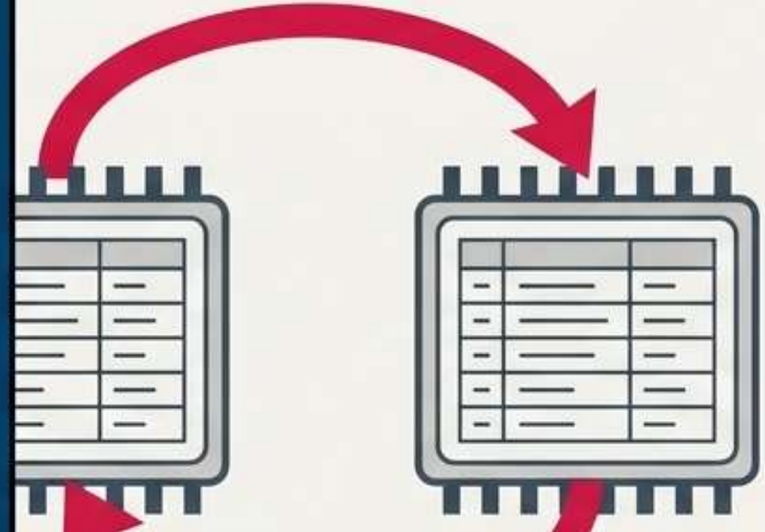
Processing Model	State Model
Active / Passive	**Active-Active Replication**



Leader computes ticks/alerts

Stan wa

Architectural Synthesis: Only one node drives the logic, both nodes hold the exact same subscription reality.



nesia RAM

Mnesia RAM

DIPARTIMENTO DI INGEGNERIA

Bridging the Runtimes

IN SUPREMA DIGNITATIS 1343

Java Environment

- Synchronous UI handling
- UI request blocking
- Mailbox PID routing

Strict JInterface Tuple Contracts

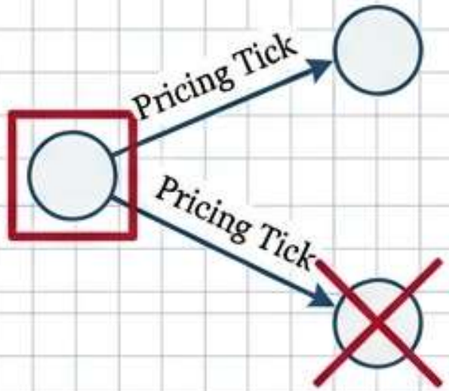
Erlang Environment

- Asynchronous core logic
- Deterministic live-node leader election
- Dynamic alert event pushes

Avoids protocol mismatch by mapping Erlang's asynchronous pushes to Java's synchronous PID-based UI routing.

Synchronization & Consistency Challenges

Leader-Side Mutex

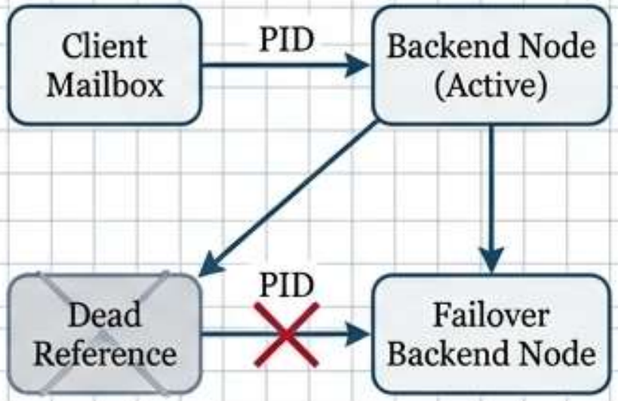


Ensuring strictly ONE backend node generates pricing ticks at a given time to prevent duplicate events.

Mnesia Race Conditions

Validating state transitions (Active -> Fired -> Removed) during reconnects and replays to prevent corrupted states.

PID Routing Validity



Ensuring client mailbox bindings remain valid after a backend failover, preventing messages from being sent to dead references.

The Subscription Lifecycle (Happy Path)

Deliver & Clean: Erlang pushes alert to Java. Java instantly broadcasts to UI. Removed from active list.

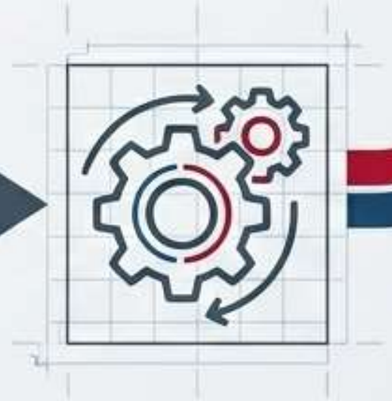
Ingest:
User sends SUB TSLA 193 via WebSockets.

Translate & Bind:
Java maps request to JInterface tuple {subscribe, ClientPid, Stock, Threshold, RawSub}.

Translate & Bind:
Java maps request to JInterface tuple

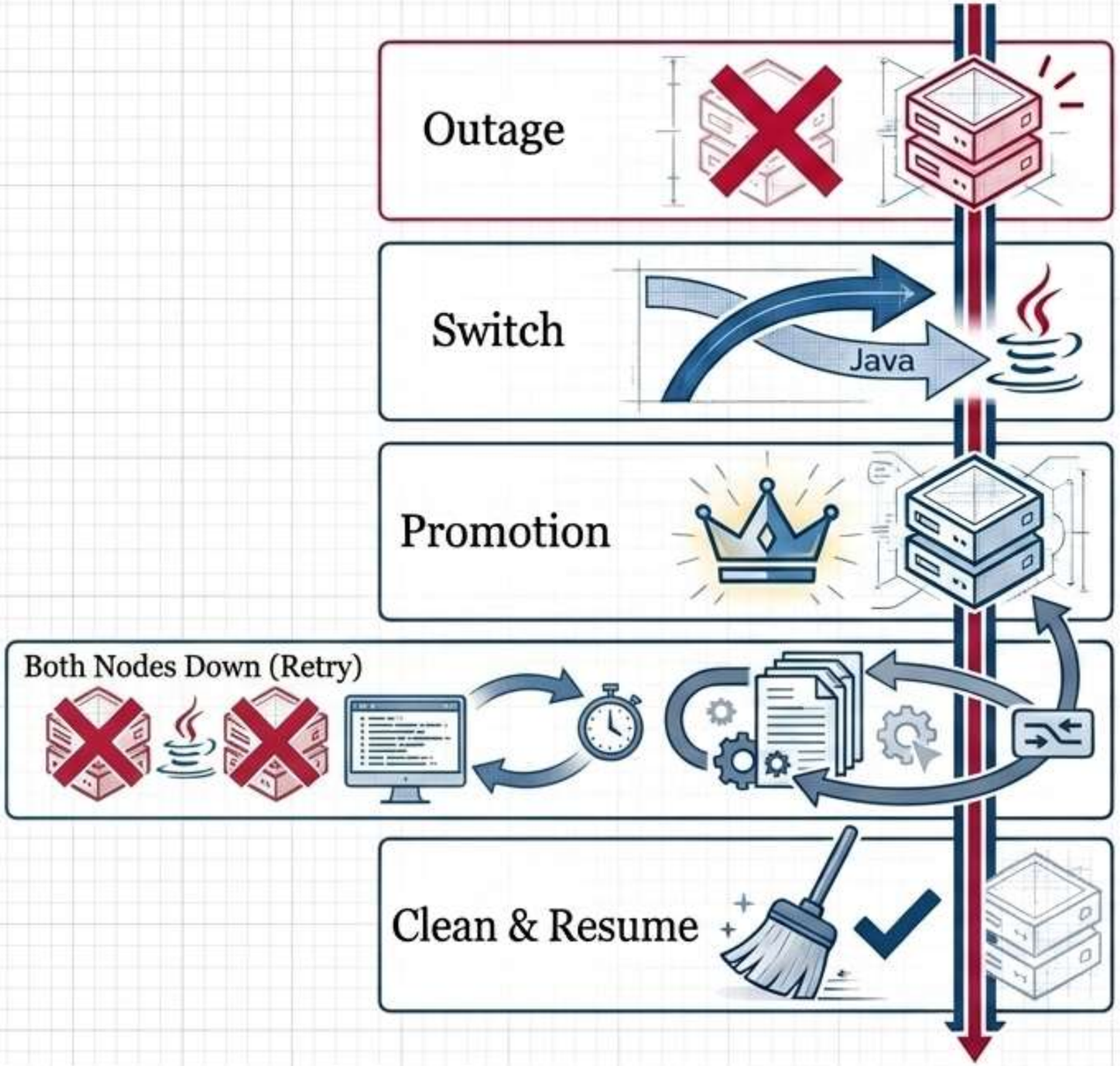
Store & Deduplicate:
Erlang stores safely in Mnesia bag table. Concurrent duplicates cleaned.

Store & Deduplicate:
Erlang stores safely in Mnesia bag table. Concurrent duplicates cleaned.



Compute:
Leader ticker updates prices. Threshold crosses.

Failover Dynamics (The Chaos Path)



1. Leader node crashes. Heartbeat mechanism detects outage.
2. Java dynamically switches backend endpoint to surviving node.
3. Standby node elevates status to the new Leader.
4. Middleware safely replays active subscriptions to the new Leader.
5. Java retries with exponential backoff and streams retry logs to the UI.
6. Stale subscriptions attached to dead node are cleaned via nodedown. Service resumes.

Deployment Strategy

Environment Specs:	Two Ubuntu containers networked over VPN.
Node Topology:	`nodeA@10.2.1.3` and `nodeB@10.2.1.14`
Security Protocol:	Shared Erlang cookie authentication for cluster joining.
State Configuration:	Mnesia explicitly set to RAM mode to optimize failover demonstration speed.
Deployment Pipeline:	Fully automated build and start via `deploy.sh` script.

Final Takeaways

Interoperability

Native **Java** <-> **Erlang** via strict **JInterface**

Distributed Coordination

Deterministic liveness & race condition elimination

Fault Tolerance

Active/Passive compute layered on Active/Active state